

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI - 590 018



A REPORT ON PROJECT WORK

Design and Analysis of Controller Area Network Protocol

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING
in
ELECTRONICS & COMMUNICATION

Project Associates

Amith A V	4BD22EC007
Chirag S	4BD22EC025
Mayuri S M	4BD22EC057
Rahul Pattar	4BD22EC069

Smt. Radhika Priya
M.Tech.MISTE.,MIETE
Project Guide

Dr. G. S. SUNITHA
M.Tech. (DEAC), Ph.D., MISTE, FIETE., FIE.
Program Coordinator

Dr. H. B. ARAVIND
M.E., (Env. Engg.) Ph.D., FIE., MISTE., FIEE.
Principal



Bapuji Educational Association®
Bapuji Institute of Engineering and Technology
Davangere-577 004
Department of Electronics & Communication Engineering
2025-26

BAPUJI INSTITUTE OF ENGINEERING AND TECHNOLOGY DAVANGERE,
KARNATAKA – 577004



DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING

Certificate

Certified that the project work entitled “**Design and Analysis of Controller Area Network Protocol**” carried out by Mr. AMITH A V, USN 4BD22EC007, Mr. CHIRAG S, USN 4BD22EC025, Ms. MAYURI S M, USN 4BD22EC057 and Mr. RAHUL PATTAR, USN 4BD22EC069, a bonafide students of this institution in partial fulfillment for the award of degree of **Bachelor of Engineering in Electronics & Communication** by **Visvesvaraya Technological University**, Belagavi during the academic year **2025-26**. It is certified that all corrections/suggestions indicated for continuous internal evaluation have been incorporated in the report.

Smt. Radhika Priya

M.Tech,MISTE.,MIETE

Project Guide

Dr. G S Sunitha

M. Tech (DEAC), Ph.D., MISTE, FIETE, FIE

Program Coordinator

Dr. H B Aravind

B.E., M.E., Ph.D., FIE, MISTE, FIEE

Principal

Internal Evaluation

Names

Signature with Date

Project Guide

Smt. Radhika Priya

.....

Faculty Member

Dr. Leela G H

.....

Program Coordinator

Dr. G S Sunitha

.....

Bapuji Educational Association®

Bapuji Institute of Engineering and Technology

Institute Vision

To be a center of excellence recognized nationally and internationally, in distinctive areas of engineering education and research, based on a culture of innovation and invention.

Institute Mission

BIET contributes to the growth and development of its students by imparting a broad-based engineering education and empowering them to be successful in their chosen field by inculcating in them positive approach, leadership qualities and ethical values.

Department Vision

To be in the forefront in providing quality technical education and research in Electronics & Communication Engineering to produce skilled professionals to cater to the challenges of the society.

Department Mission

M1: To facilitate the students with profound technical knowledge through effective teaching learning process for a successful career.

M2: To impart quality education to strengthen students to meet the industry Standards and face confidently the challenges in the programmed.

M3: To develop the essence of innovation and research among students and faculty by providing infrastructure and a conducive environment.

M4: To inculcate the student community with ethical values, communication Skills, leadership qualities, entrepreneurial skills and lifelong learning to meet the societal needs.

PROJECT WORK

Course Learning Objectives:

This course will enable students to:

1. Prepare with foundation knowledge in a project domain through designing, and implementing.
2. Introduce to various emerging fields in electronics and communication.
3. Provide an opportunity to exercise the creative and innovative qualities, to become technopreneurs in group project environment.
4. Have hands-on experience in the related field so that students can integrate and reinforce what has been taught in the classroom.

Abstract

This project focuses on the design and simulation of a CMOS Controller Area Network (CAN) transceiver implemented using the Cadence Virtuoso design environment based on a 180 nm CMOS process. The aim is to develop a low-power, high-speed, and robust communication interface that meets the ISO-11898 CAN bus standard, ensuring reliable data transmission in noisy industrial and automotive environments. The proposed transceiver integrates both transmitter and receiver circuits on a single chip, emphasizing signal integrity, EMI reduction, and thermal reliability. The transmitter section converts digital input data from the CAN controller (TXD) into differential output signals (CANH and CANL) suitable for bus communication.

ACKNOWLEDGEMENT

We would like to acknowledge the help and encouragement given by various people during the course of this project.

We are deeply indebted and very grateful for the invaluable guidance provided by our project guide, **Smt. Radhika Priya Y.R.**, Assistant Professor, throughout the course of this work.

We would like to thank our project coordinators, **Dr. Leela G. H.**, Professor, and **Mr. Yogesha K. O.**, Assistant Professor for their kind coordination and support during the project.

We express our sincere gratitude to **Dr. G. S. Sunitha**, Professor and Head, Department of Electronics and Communication Engineering, BIET, Davanagere, for her support, guidance, and constant encouragement throughout the duration of the project.

We are thankful to our beloved Principal, **Dr. H. B. Aravind**, for providing an excellent academic environment that greatly facilitated our project work.

We also extend our thanks to all the teaching and non-teaching staff of the Department of Electronics and Communication Engineering for their cooperation and assistance during the project. The support extended by the college and departmental library and laboratory is also gratefully acknowledged.

Finally, we are deeply thankful to our parents and friends, whose encouragement and help in various ways have been invaluable throughout the course of our project work.

Project Associates

Amith A V

Chirag S

Mayuri S M

Rahul Pattar

List of Contents

Chapter 1	Introduction	1
Chapter 2	Literature Survey	2
Chapter 3	Problem Identification	6
Chapter 4	Objectives	7
Chapter 5	Block Diagram	
Chapter 6	Methodology	8
Chapter 7	Software Requirement	12
Chapter 8	Design Implementation	15
	8.1 Stage 1: Input Stage	15
	8.2 Stage 2: Middle Stage	16
	8.3 Stage 3: Can_H and Can_L Stage	17
	8.4 Stage 4: Receiver stage	21
Chapter 8	Results and Discussion	23
	Application	26
	Advantages	26
	Limitations	26
	Conclusion	28
	Future Scope	28
	References	29
	Course Outcomes (CO)	30
	Contact Details	31

List of Figures

Figure 5.1	Block Diagram	10
Figure 7.1	Cadence Tool	12
Figure 7.2	Virtuoso Analog Design Environment Window	13
Figure 8.1.1	Circuit for Input Stage of transmitter	15
Figure 8.1.2	Schematic design for Input Stage	15
Figure 8.2.1	Circuit for Middle stage of transmitter	17
Figure 8.2.2	Schematic design for Middle stage	17
Figure 8.3.1	Circuit for Can_H and Can_L Stage	19
Figure 8.3.2	Schematic design for Can_H and Can_L Stage	19
Figure 8.4.1	Circuit for Receiver Stage	21
Figure 8.4.2	Schematic design for Receiver stage	21

CHAPTER 1

INTRODUCTION

The Controller Area Network (CAN) is a robust serial communication protocol developed by Robert Bosch GmbH in 1986, primarily for real-time data exchange in automotive applications. Over time, it has evolved into a global standard, ISO 11898, due to its high reliability, real-time capability, and fault tolerance. The CAN protocol is widely used in automotive electronics, industrial automation, robotics, and medical instrumentation, enabling multiple microcontrollers and sensors to communicate efficiently without the need for a central host computer.

A CAN transceiver acts as the physical interface between the CAN controller and the CAN bus lines. It plays a crucial role in converting logic-level signals from the microcontroller into differential signals that can be transmitted over the twisted-pair bus, and vice versa. The differential signaling technique enhances the system's noise immunity, allowing reliable data transmission in electrically noisy environments.

This project presents the design and simulation of a CMOS-based CAN transceiver with slew rate control, implemented in Cadence Virtuoso using a 180 nm CMOS process technology. The 180 nm node is chosen for its optimal balance of speed, cost efficiency, and power consumption, making it suitable for low-cost, high-performance mixed-signal designs.

The proposed transceiver consists of two major sections — transmitter and receiver.

- The transmitter converts digital input signals (TXD) into differential outputs (CANH and CANL) for the CAN bus. It comprises an input stage, middle stage, slew rate control circuit, and output stage. The slew rate control feature allows dynamic adjustment of signal transition speed through an external resistor (R_s), thereby minimizing electromagnetic interference (EMI) and supporting various communication speeds.
- The receiver is designed as a hysteresis comparator that converts differential input signals from the bus into logic-level outputs (RXD) for the CAN controller. The hysteresis characteristic significantly enhances noise rejection and ensures stable signal recovery even in high-interference environments.

This project aims to achieve a fully functional, low-power, and EMI-resistant CAN transceiver with adjustable slew rate, providing an efficient and reliable physical layer solution for next-generation automotive and industrial communication systems.

CHAPTER 2

LITERATURE SURVEY

Paper 1

Title: *Design of the Circuits for a CMOS CAN Transceiver Chip with Slew Rate Control*

Author(s): Huihui Zhan, Shaoxin Zong, Xiugang Han, Chuannan Li

Year of Publication: 2012

Technology Used: 0.5 μ m n-well CMOS process, HSPICE simulation

Keywords: CAN bus, transceiver, CMOS process, slew rate control

Methodology: A complete CAN transceiver circuit was designed using CMOS technology, with both transmitter and receiver circuits. The transmitter comprises input, middle, slew-rate control, and output stages. The middle stage uses cascaded inverters for high drive strength, while slew rate control allows EMI tuning through external resistance. The receiver features a hysteresis comparator for noise resilience. This paper presents a CMOS CAN transceiver design optimized for low power and high speed with integrated protections and controllable slew rates. The architecture enhances signal symmetry, noise immunity, and adaptability for various data rates. Simulations confirm ISO 11898 compliance and 1 Mbit/s operation, validating its suitability for robust automotive and industrial use.

Paper 2

Title: *Secure Controller Area Network (CAN) Transceiver With Embedded Authentication*

Support Author(s): Weizhong Chen, Xianshan Wen, Can Hong, Theodore W. Manikas, Mitchell A. Thornton, Ping Gui

Year of Publication: 2025 (Accepted by IEEE)

Technology Used: 0.18 μ m CMOS process, mixed-signal design

Keywords: Automotive security, CAN transceiver, frame-level authentication, message authentication code, dual-rail converter

Methodology: A novel CAN transceiver design introduces an embedded virtual channel to support frame-level authentication via phase modulation of signal edges. Hardware Security Modules (HSMs) generate and verify authentication codes. Synchronization and phase extraction modules recover authentication data without altering CAN frame structure, ensuring backward compatibility.

Paper 3

Title: *An Integrated Analog-CMOS Control System for Low-Cost Applications*

Author(s): Daniel Batas, Klaus Schumacher

Year of Publication: 2002

Technology Used: Analog CMOS, switched capacitor technique

Keywords: Analog-CMOS, motor control, low-cost, parametric field, current-controlled oscillator

Methodology: The paper proposes a fully integrated analog control system using a CMOS process to minimize cost and board area for motor control. A programmable analog parametric field acts as the control logic. A current-controlled oscillator (CCO) regulates power delivery to the motor, while feedback and switching control is implemented through on-chip analog circuitry and switched capacitor filtering. This paper introduces a compact, low-cost analog CMOS system designed to replace microcontroller-based motor controllers. By integrating analog signal paths and programmable control via a serial interface, the system provides continuous-time signal handling without discretization. Features include a power supply, zero-cross detector, motor logic, power amplifier, and AC-DC conversion using minimal external components. The design is ideal for cost-sensitive embedded motor applications.

Paper 4

Title: Design and Verification of a CAN Controller for Custom ASIC

Author(s): Namhoon Kim, Daejong Kim, Kwangyong Cho, Jonghoon Kim, Wonho Cho

Year of Publication: 2005

Technology Used: Verilog HDL, ASIC, RTL design and verification

Keywords: CAN controller, RTL design, verification, custom ASIC, protocol validation

Methodology: The paper presents the design and functional verification of a CAN protocol controller intended for integration into a custom ASIC. The CAN controller is modeled in Verilog at the RTL level and supports standard CAN 2.0A/B features such as bit timing, arbitration, error detection, and frame handling. The authors implemented a testbench that simulates multiple frames, arbitration conditions, and bus states.

Functional simulations validate the controller's adherence to the protocol, while synthesis confirms resource efficiency. A compact and protocol-compliant CAN controller is developed and verified for use in embedded SoC designs. The work emphasizes early verification and error-handling logic to meet automotive safety and timing constraints, demonstrating the potential for integrating CAN logic in ASIC workflows.

Paper 5

Title: Design Method of CAN Bus Network Communication Structure for Electric Vehicle

Author(s): Lei Ran, Wang Junfeng, Wang Haiying

Year of Publication: 2010

Technology Used: Simulation (network topology), CAN protocol, MATLAB/Simulink

Keywords: CAN bus, electric vehicle, communication structure, node optimization

Methodology:

This paper investigates the optimal communication structure for implementing a CAN bus in electric vehicles (EVs). It models and simulates a hierarchical CAN topology in MATLAB/Simulink, considering message priority, bandwidth, and real-time constraints. The design separates high-speed control (e.g., motor and battery management) and low-speed infotainment communication. It also includes arbitration delay analysis and message scheduling to ensure deterministic delivery. The work provides an efficient design strategy for CAN-based communication in electric vehicles. It balances communication loads across multiple nodes, reducing latency while maintaining modularity and scalability in EV control systems.

Literature Survey Summary

Ref. Paper Authors/Year	Objectives / Issues Addressed	Parameter Evaluated	Adaptive Technique	Drawbacks	Open Issue / Improvements
Huihui Zhan et al. (2012)	To design a full-custom CMOS CAN transceiver with slew rate control and protection features.	Slew rate, delay, drive strength, temperature tolerance.	Slew rate control via Rs and current mirrors.	Focused only on analog layer; lacks digital CAN controller integration.	Combine with digital CAN core to create complete SoC design.
Shinde et al. (2014)	Implementation of CAN Bus protocol using low-cost microcontrollers.	Latency, node communication integrity.	SPI communication using MCP2515 module with microcontrollers.	Not scalable to ASIC or SoC; limited to MCU-based systems.	Add EMI and fault tolerance features; port design to hardware.
Namsub Kim et al. (2005)	To design and verify a CAN controller for ASIC with ISO conformance and reduced gate count.	Gate count, error detection, conformance testing.	Modular verification approach with FIFO buffer architecture.	Limited flexibility in legacy CAN mode; complex verification.	Extend to CAN FD mode; reuse modular IP with AI-driven arbitration.
Li Ran et al. (2010)	To develop a design method for CAN Bus network used in electric vehicles.	Node mapping, frame integrity, transmission stability.	SAE J1939 extension and ID mapping table.	Lacks physical layer or circuit-level noise simulation.	Add PHY-layer modeling for EMI and noise robustness.
Pardeep Kaur, Chanpreet Toor (2017)	To implement CAN protocol on FPGA for real-time automotive communication.	Response time, bit count, deadline alignment.	Dynamic ID window for arbitration and scheduling.	Does not include analog transceiver or physical layer modeling.	Integrate with analog driver/transceiver for full CAN stack implementation.

Chapter 3

Problem Identification

1. **Excessive Wiring in Traditional Systems**

Point-to-point communication caused bulky and complex wiring, resulting in increased vehicle weight and higher costs.

2. **Inefficient Communication Methods**

There was no standardized method for reliable multi-device communication. Existing methods lacked both speed and robustness.

3. **Poor Fault Tolerance**

Legacy protocols offered limited error detection and were not suitable for noisy or safety-critical environments.

4. **Scalability Limitations**

It was difficult to integrate new devices into existing networks, as this required major hardware modifications.

5. **Lack of Real-Time Support**

Time-critical systems (e.g., engine control) require deterministic data transmission, but older protocols failed to meet real-time performance requirements.

Chapter 4

Objectives

- Design a transmitter and receiver circuit for the CAN protocol at the transistor level using CMOS technology.
- To provide strong protection mechanisms, including over-voltage, under-voltage, short-circuit, and thermal shutdown protection, ensuring reliability in harsh environments.
- To develop a synthesizable RTL design in the Cadence environment that generates accurate CAN_H and CAN_L differential waveforms, enabling verification of the CAN physical-layer behavior at the digital level.

CHAPTER 5

BLOCK DIAGRAM

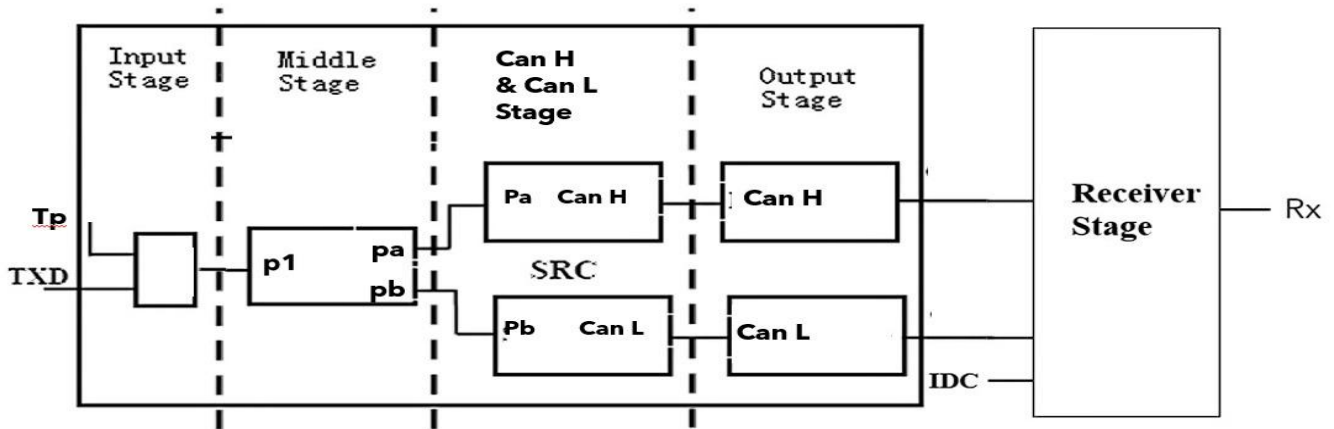


Figure 6.1: Block Diagram

The above figure illustrates the **block diagram of the CAN Transmitter Circuit**, which plays a crucial role in converting **digital signals (TXD)** from the CAN controller into **differential output signals (CANH and CANL)** for communication over the CAN bus. The design ensures robust, high-speed, and noise-immune data transmission between nodes in an automotive or industrial CAN network.

Design Description

The transmitter circuit consists of **three major stages** — the **Input Stage**, the **Middle Stage**, and the **Slew Rate Control (SRC) Output Stage** — each performing a specific function to ensure reliable signal conversion and protection of the circuit.

1. Input Stage:

The input stage receives a digital signal (TXD) from the CAN controller, which can be either a **TTL** or **CMOS logic signal**. This stage acts as a **signal conditioning circuit**, converting the TXD input into appropriate voltage levels suitable for the next stages. It also includes internal protection features against electrostatic discharge (ESD) and voltage spikes that may occur in harsh automotive environments.

2. Middle Stage:

The middle stage, also referred to as the **driver or control stage**, is responsible for amplifying and controlling the logic signals. It receives the processed input and generates corresponding differential signals. This stage ensures that the output voltage levels are properly driven to match the required bus characteristics. It also manages transitions between dominant and recessive states of the CAN bus.

3. CANH and CANL and Output Stage:

The slew rate control circuit determines the **speed of the signal transition** between logic states on the CAN bus. By controlling the rise and fall times of the output signals, the SRC circuit effectively minimizes electromagnetic interference (EMI) and ensures signal integrity, especially over long communication lines. The final outputs, , are driven in opposite directions to produce the differential signal pair.

The **Controller Area Network (CAN) bus** is a robust, message-based communication protocol designed primarily for in-vehicle networking, though its reliability has led to its wide adoption in industrial and medical fields. It was developed by Bosch to replace complex, heavy point-to-point wiring with a simple, shared two-wire network. The entire communication system hinges on **differential signaling** over these two wires: **CAN-High** and **CAN-Low**. The voltage difference between CAN_H and CAN_L determines the logical state of the bus, which provides exceptional **noise immunity** in electrically noisy environments. The two wires operate in one of two states: the **Dominant** state (representing a logical '0'), where CAN_H is driven high e.g., 3.5V and CANL is driven low (e.g., 1.5V, resulting in a differential voltage of about 2V; and the **Recessive** state (representing a logical '1'), where both lines are nominally at the same voltage (e.g., 2.5V), resulting in a 0V differential. This differential approach is key to CAN's robustness and efficiency, as the CAN transceiver (which you are designing) is responsible for converting the single-ended digital signals TXD, RXD to and from these differential CANH and CANL bus levels.

CHAPTER 6

METHODOLOGY

The design and simulation of the **CMOS CAN Transceiver with Slew Rate Control** were carried out using the **Cadence Virtuoso design environment** based on the **180 nm CMOS technology**. The process followed systematic stages—from specification analysis to layout verification—ensuring performance, reliability, and compliance with the **CAN (Controller Area Network) standard ISO 11898**.

1. System Specification and Design Objectives

The transceiver was developed to enable **high-speed, noise-immune differential communication** between CAN nodes. Design objectives included:

- Operation speed up to **1 Mbps**
 - **Differential signaling** for high noise immunity
 - **Slew-rate control** via an external resistor to reduce EMI
 - Built-in **protection circuits** for overvoltage, undervoltage, and short-circuit conditions
 - **Low power consumption** with stable performance across temperature variations
- The **180 nm CMOS process** was chosen for its balanced trade-off between speed, area efficiency, and device reliability for mixed-signal designs.

2. Transmitter Circuit Design

The **transmitter** converts the digital input (TXD) into differential outputs (CANH and CANL). It includes:

- **Input Stage:** Accepts TXD and thermal shutdown signal (T_p). When temperature exceeds 150 °C, T_p disables the driver to prevent damage.
- **Middle Stage:** Comprises five CMOS inverters with increasing transistor sizes to drive large capacitive loads.
- **Slew Rate Control (SRC):** Uses current mirrors and an external resistor (R_s) to regulate the charging/discharging rate of the output transistors, thereby controlling transition speed and EMI.
- **Output Stage:** Large MOSFETs provide strong drive capability, while series resistors and Schottky diodes offer protection and ensure differential balance.

3. Receiver Circuit Design

The receiver detects differential voltages from CANH and CANL and regenerates the logic output (RXD). It consists of a two-stage hysteresis comparator:

- **Differential input pair** amplifies small voltage differences and rejects common-mode noise.
- **Positive feedback** adds hysteresis for noise immunity.
- **Output buffer** restores logic levels to CMOS standards.

4. Implementation in Cadence Virtuoso

The entire circuit was designed in **Cadence Virtuoso Schematic Editor**, and simulations were executed using **Spectre Simulator**. Analyses included:

- **DC analysis** for bias point verification
- **Transient analysis** for slew rate and data rate validation
- **Temperature sweep** (−40 °C to 125 °C) for thermal stability
- **Parametric sweep of Rs** to evaluate slew-rate control
Post-layout simulations with **RC parasitic extraction** validated real-world performance under process variations.

5. Layout and Verification

The layout adhered to **180 nm design rules**, ensuring symmetry between CANH and CANL lines and minimal parasitics. **DRC** (Design Rule Check) and **LVS** (Layout vs. Schematic) confirmed design integrity. The final layout was optimized for compact area, signal integrity, and reliable

CHAPTER 7

SOFTWARE REQUIREMENTS

Cadence Virtuoso is a comprehensive suite of software tools used for electronic designautomation (EDA), particularly for the design and verification of integrated circuits (ICs) and related systems. It encompasses various components, including layout, schematic, simulation, and verification tools, enabling designers to create and analyze complex designs from schematic entry to physical layout.



Figure 7.1: Cadence

7.1 Overview of the Tool: Cadence Virtuoso

Cadence Virtuoso offers a complete suite for full-custom IC design, including schematic entry, simulation setup, waveform analysis, and layout generation. The environment supports SPICE-level simulations using the **Spectre simulation engine**, which delivers high-accuracy results for both analog and mixed-signal designs. The platform also includes parameter sweep, corner analysis, and Monte Carlo simulation features, making it ideal for characterizing variations in CMOS performance due to fabrication mismatches.

7.2 Key Modules and Features Utilized

In the development and simulation of the CMOS-based R-2R DAC and the associated two-stage operational amplifier, several key modules within Cadence Virtuoso were extensively used. Each module played a crucial role in building, analyzing, and verifying the design at the transistor level:

7.2.1 Virtuoso Schematic Editor

This module was used to construct the circuit schematic for both the two-stage op-amp and the R-2R DAC. It provides a user-friendly interface for placing NMOS and PMOS transistors, resistors, capacitors, voltage sources, and input/output pins.

Each component was configured with proper dimensions (W/L ratio for transistors) and connected to form a complete analog subsystem supports hierarchical design, allowing the op-amp to be built as a reusable block in the DAC.

7.2.2 Virtuoso Analog Design Environment (ADE)

ADE is the control center for simulations in Cadence. It allows the user to set up different types of analyses (DC, AC, Transient), define input sources, select output signals to be plotted, and configure simulation runs

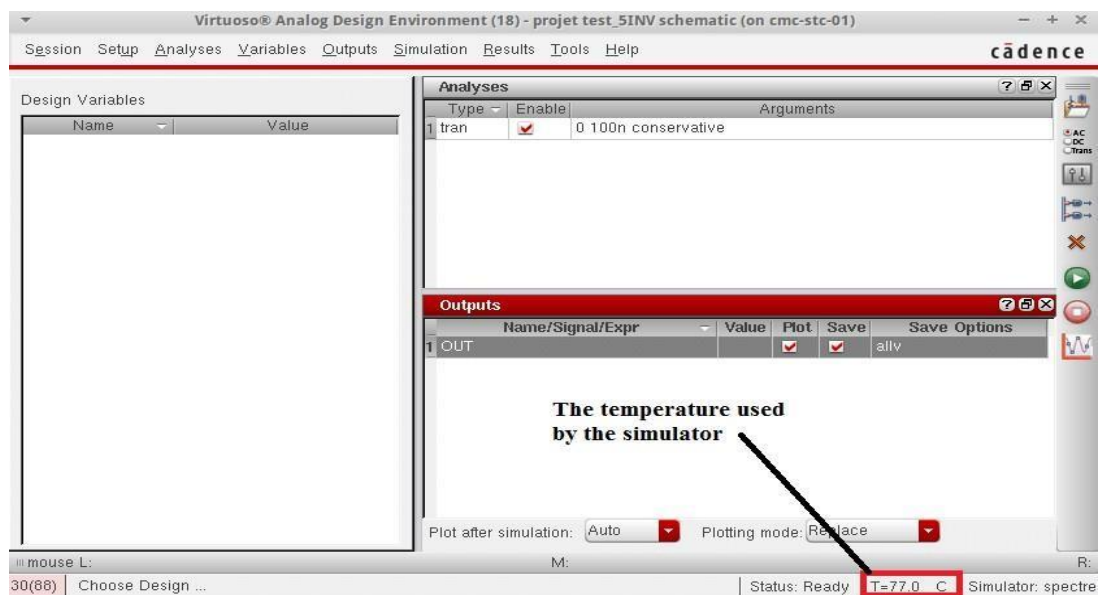


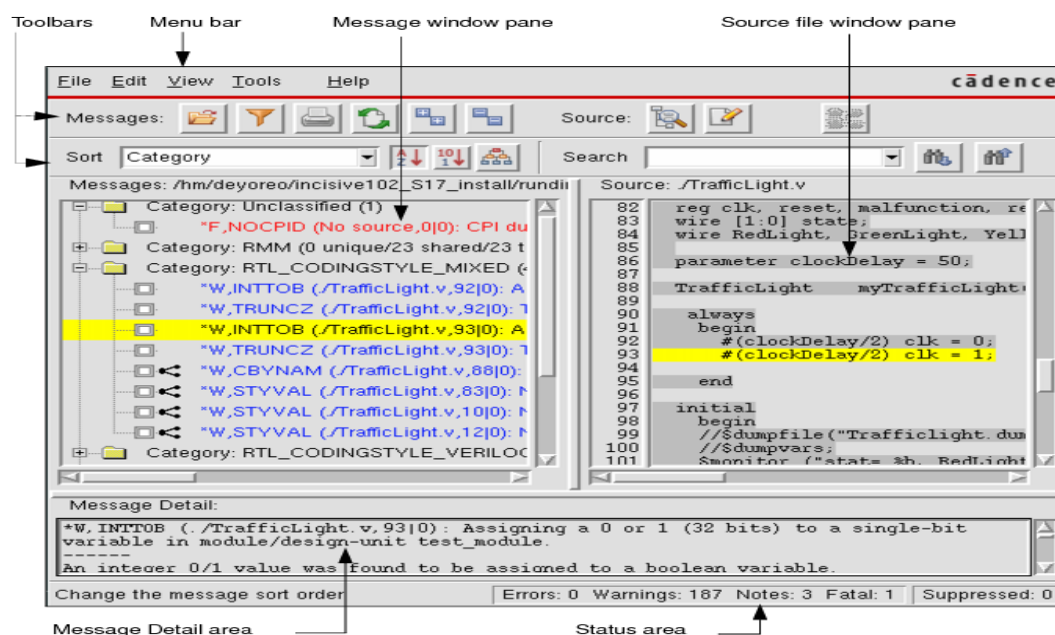
Figure 7.2: Virtuoso Analog Design Environment Window

For this project, ADE window shown in fig. 6.2.2.1 was used to:

- Perform **DC operating point analysis** to verify node voltages and current paths.
- Execute **AC analysis** to observe gain and phase responses of the op-amp.
- Conduct **Transient analysis** to simulate step responses and signal transitions in the DAC output.
- Automate **parameter sweeps** (e.g., input voltage or bias current) for performance tuning.

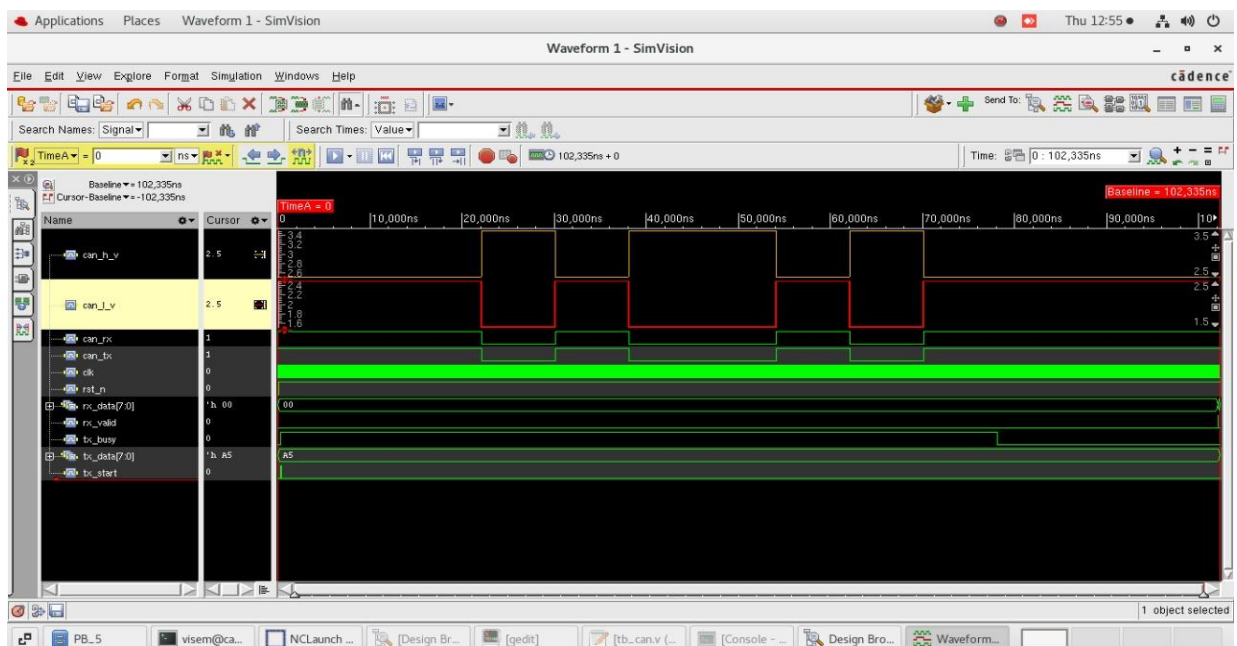
7.3 NC Launch description

NC Launch is the graphical interface of the Cadence Incisive simulator used in this project to compile, elaborate, and simulate the complete CAN controller design. All RTL modules, including the bit-timing, transmitter, receiver, CRC unit, buffer unit, and the top-level CAN controller, were added and compiled through NC Launch to generate a simulation snapshot. The tool was then used to run the testbench, analyze signal behavior, and debug the design using the integrated SimVision waveform viewer. This environment provided an efficient and organized flow for validating functional correctness, protocol timing, and internal state transitions of the CAN system.



RTL DESCRIPTION

The CAN controller RTL design consists of several modular Verilog components that together implement a complete CAN 2.0A/B communication system. The **crc15.v** module computes the 15-bit CRC required for CAN frame error detection using a polynomial-based shift register architecture. The **can_bittiming.v** block generates all essential timing signals, including programmable baud rate prescaler (BRP), synchronization jump width (SJW), TSEG1/TSEG2 segments, and the sample point needed for both transmission and reception. The **can_tx.v** module implements the CAN transmitter finite state machine, handling arbitration, data/remote frame formatting, stuffing bits, CRC generation, ACK slot processing, and EOF generation. Complementing this, **can_rx.v** performs frame reception by synchronizing incoming bits, performing de-stuffing, checking CRC, decoding identifiers and data fields, and validating ACK and error flags. The **can_tbu.v** (Transmit Buffer Unit) stores outgoing messages, manages data buffering, and provides arbitration support to the transmitter. All these components are integrated in **can_top.v**, which serves as the system-level wrapper connecting the bit-timing unit, TX/RX engines, CRC logic, buffers, and control/status interfaces. This top module also generates internal interrupts and manages frame-level handshaking. The associated **tb_can.v** testbench instantiates the CAN top module, applies timing-accurate stimulus, drives random frames, monitors TX/RX waveforms, and verifies protocol correctness through simulation. Supporting notes and simulation logs recorded in **gedit.txt** help maintain documentation and track design changes. Overall, the RTL architecture is modular, synthesizable, and structured to accurately model the complete behavior of a CAN controller in hardware.



CHAPTER 8

DESIGN IMPLEMENTATION

Stage 1: Input stage

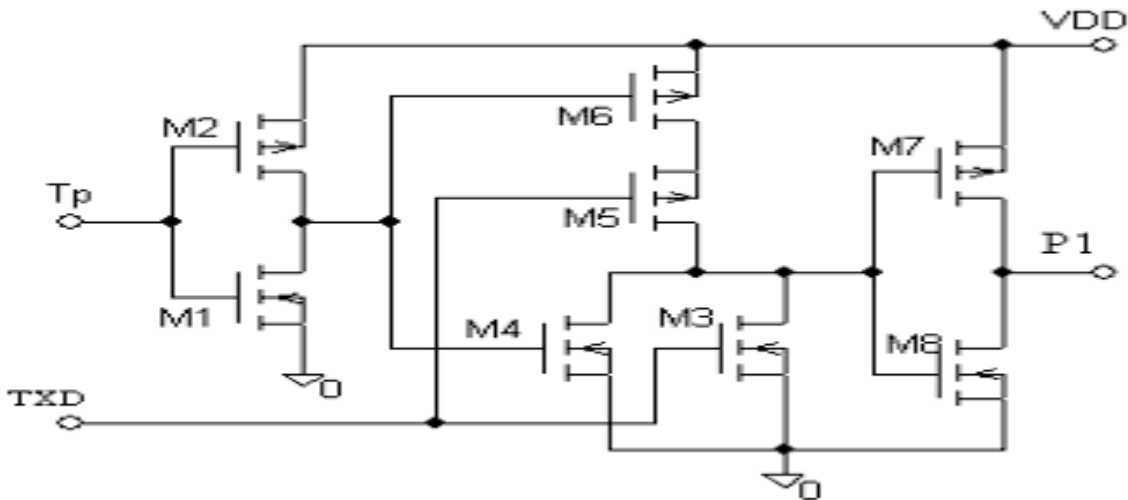


Figure 8.1.1: Circuit for Input Stage of the transmitter

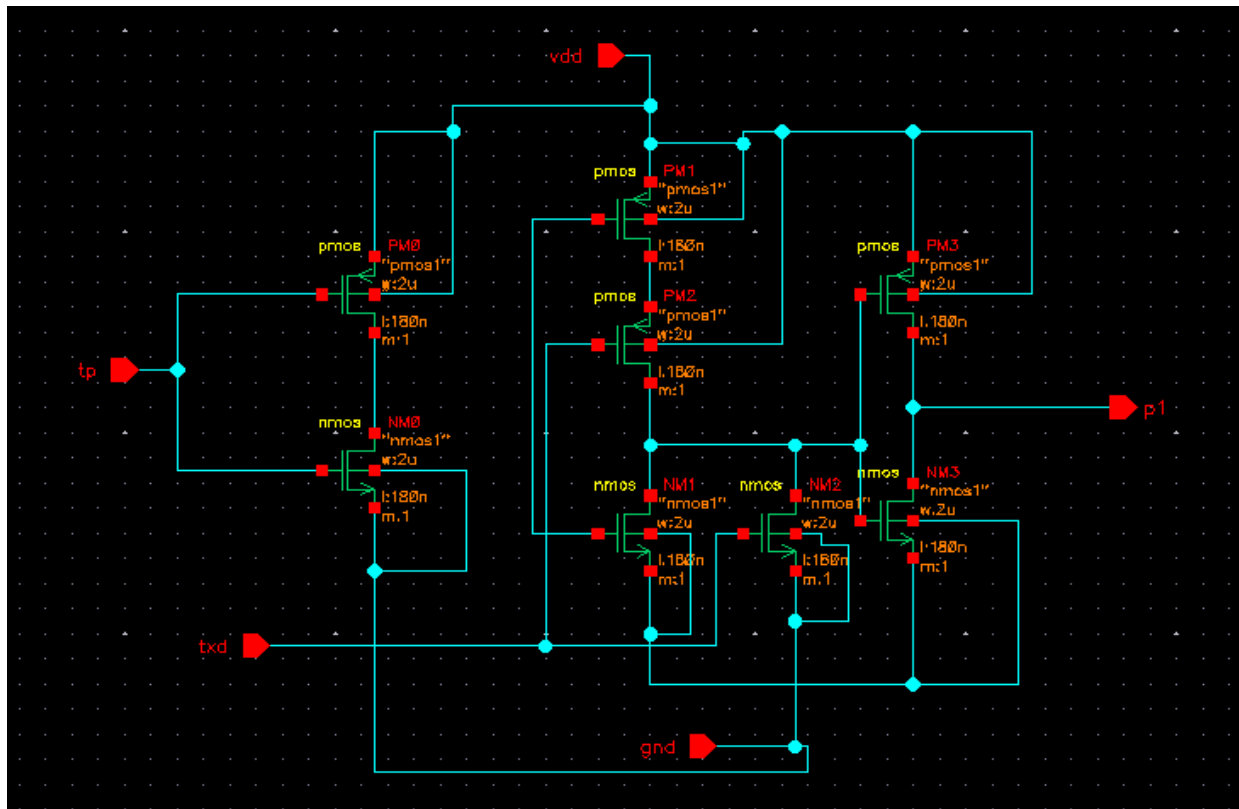


Figure 8.1.2: Schematic design Input Stage

The circuit is built using eight MOS transistors (M1–M8) arranged to perform logic control and temperature-based shutdown. The T_p signal is generated by the internal thermal protection block, which monitors the chip's temperature.

When the chip temperature exceeds approximately 150°C, T_p goes low, forcing the circuit output to a constant logic “1,” thereby disabling the transmitter's switching activity.

When the temperature drops back below around 125°C, T_p returns to a high level, re-enabling normal transmitter operation.

The TXD input controls the data flow. When TXD = ‘0’ (logic low), transistors M3 and M4 conduct, pulling the output node toward ground. When TXD = ‘1’ (logic high), M5 and M6 turn on, driving the output toward VDD, which represents a logic “1.” Transistors M7 and M8 form an additional inverter stage that provides buffering and stable output (P1) for the next stage of the transmitter (middle stage)

The transistor pair M1–M2 functions as a switch controlled by T_p , which effectively isolates or enables the main inverter network depending on the temperature condition. When T_p is high, M1 is off and M2 is on, allowing the TXD signal to propagate through the logic network. When T_p is low, both transistors disable the logic chain, maintaining the output in a fixed high state.

Overall, this input stage performs two critical functions — (1) translating the digital TXD signal into a clean CMOS-compatible level for the subsequent stages, and (2) providing thermal shutdown protection, thereby enhancing circuit reliability and reducing power consumption during fault or over-temperature conditions.

Stage 2: Middle Stage

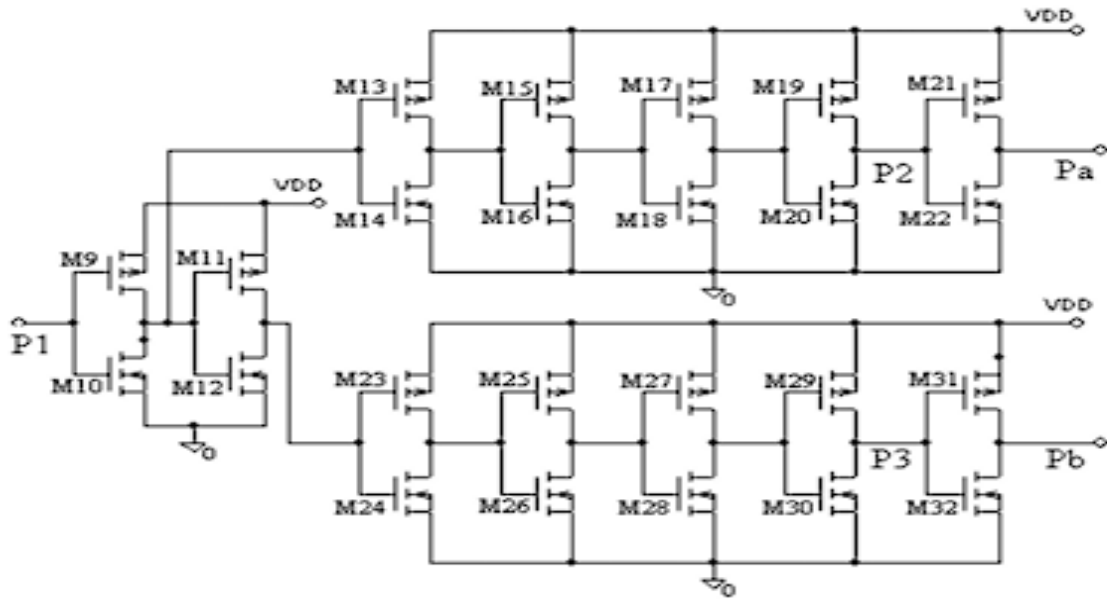


Figure 8.2.1: Circuit for Middle Stage of transmitter

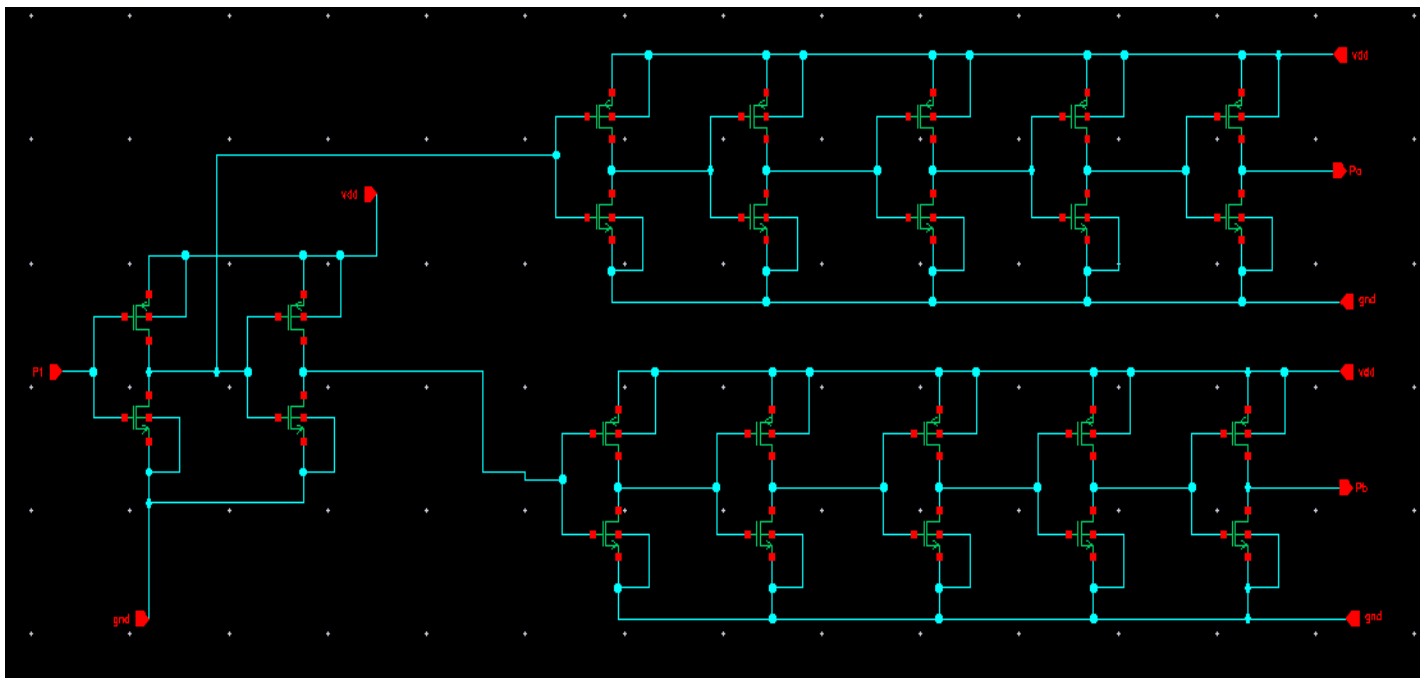


Figure 8.2.2: Schematic Design for Middle Stage

The middle stage of the CAN transmitter acts as a signal driver and phase-inversion network, converting the single-ended input signal (P1) from the input stage into two complementary outputs (Pa and Pb). This stage is crucial for driving the large gate capacitances of the high-power output transistors (M40 and M41) in the next stage while maintaining high-speed operation and minimal delay.

The circuit mainly consists of five cascaded CMOS inverter pairs in each branch — the upper branch (M13–M22) and the lower branch (M23–M32) — which together generate two signals of opposite polarity. The transistors are arranged in a progressive sizing pattern, where each successive inverter is designed with a larger width-to-length (W/L) ratio than the previous one. This scaling ensures a higher driving current and a sharper transition, enabling the circuit to switch the large load capacitances quickly and efficiently.

Transistors M9–M12 form a differential input stage that splits the incoming signal P1 into two paths. These control the two identical inverter chains that produce complementary outputs. The upper inverter chain produces the non-inverted output Pa, while the lower inverter chain produces the inverted output Pb. Each chain consists of alternating PMOS and NMOS transistors, providing rail-to-rail swing and ensuring symmetrical rise and fall times.

This configuration enhances the overall driving capability, reduces propagation delay, and maintains waveform integrity at high frequencies. The use of five cascaded inverters in each path minimizes skew between the two complementary outputs and ensures that both Pa and Pb signals are perfectly synchronized but 180° out of phase.

Stage 3: Can_H and Can_L Stage

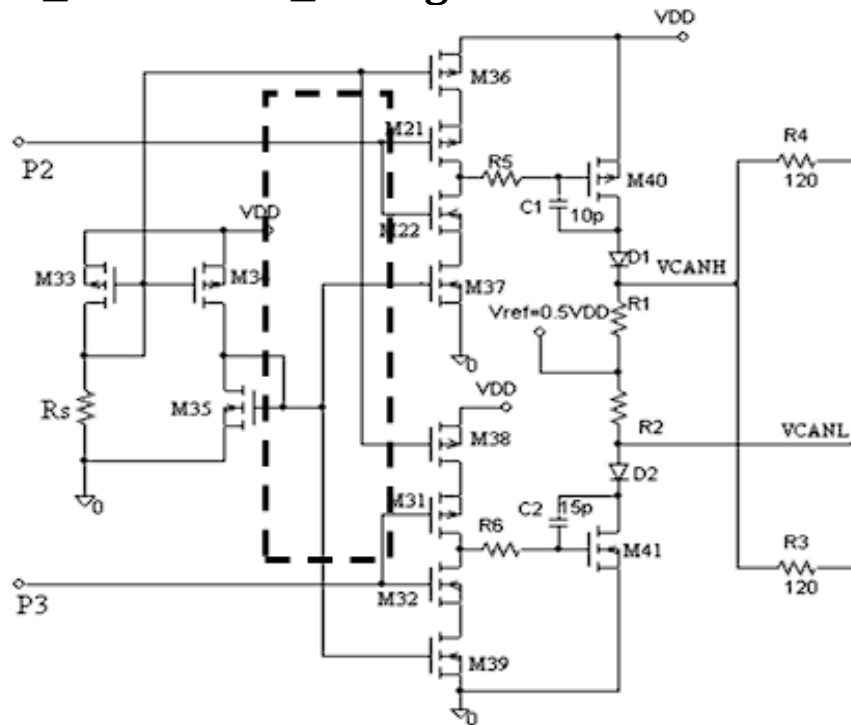


Figure 8.3.1: Circuit for Can_H and Can_L Stage

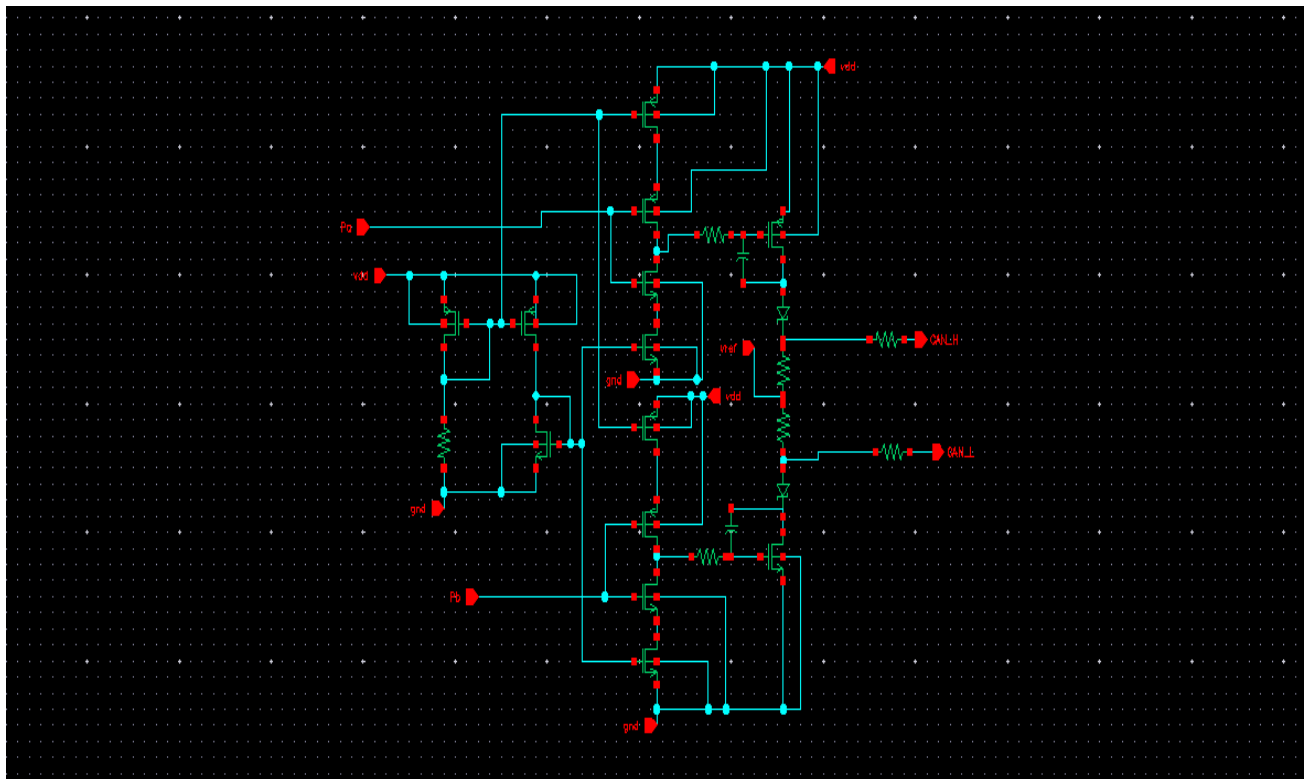


Figure 8.3.2: Schematic Design for can_H and Can_L Stage

The transistors M33–M39 form the core signal-driving network that conditions the differential outputs for CAN_H and CAN_L. These transistors regulate the flow of current toward the output driver devices and ensure proper shaping of the signals delivered to the bus. The external resistor R_s establishes the biasing current for M33–M35, which act as current-controlling devices to regulate the charging and discharging activity inside this driver section.

The differential control signals P2 and P3, received from the middle stage, are applied to this circuit to drive the large output transistors. Capacitors C1 (10 pF) and C2 (15 pF) are connected across the gate nodes of M40 and M41 to improve stability, reduce noise sensitivity, and maintain consistent gate-voltage behavior during transitions.

At the output, the driver transistors M40 and M41 generate the complementary differential voltages **VCANH** and **VCANL**, which form the actual CAN bus signals. Resistors R1 and R2 operate as current-limiting resistors, protecting the output devices during switching. Schottky diodes D1 and D2 provide overvoltage and undervoltage protection by clamping the output nodes and preventing reverse current flow when bus voltages exceed or fall below safe operating limits. The **120 Ω resistors** (R3 and R4) represent the standard CAN bus termination impedance, used to match the characteristic line impedance and minimize signal reflections.

A stable reference voltage (**Vref = 0.5 VDD**) is applied to maintain the proper common-mode level of the differential outputs. Together, these components ensure balanced and reliable CAN_H and CAN_L signals that comply with CAN physical-layer requirements.

Stage 4: Receiver Stage

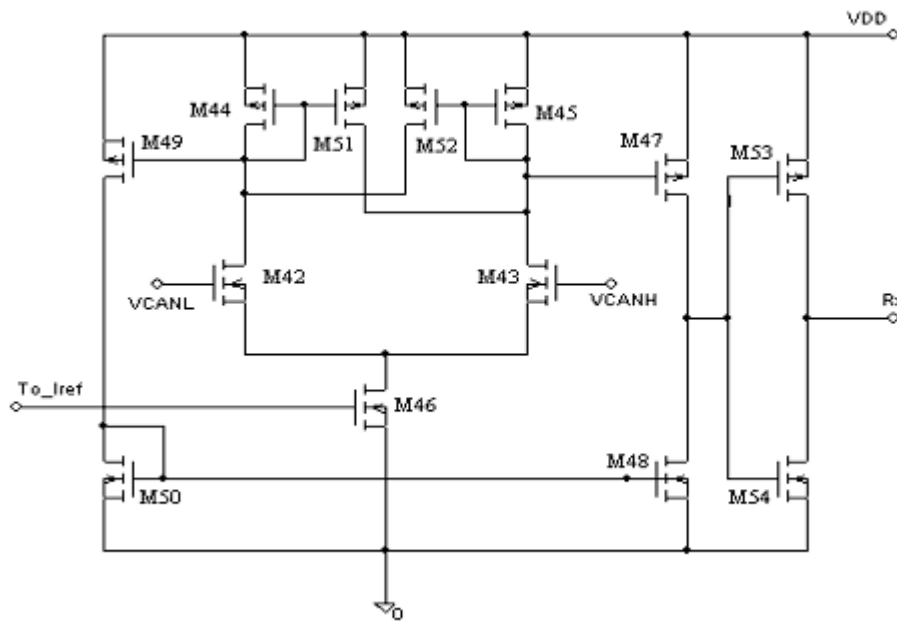


Figure 8.4.1: Circuit for Receiver stage

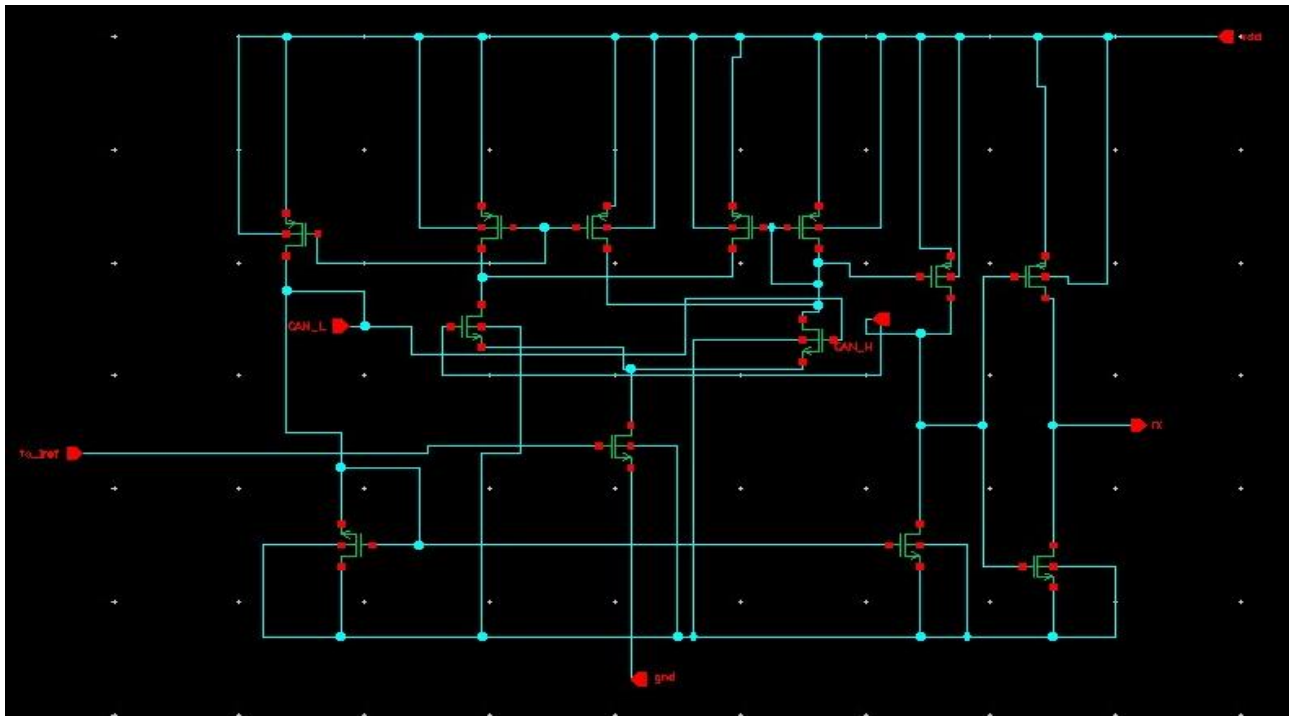


Figure 8.4.2: Schematic Design for Receiver stage

The receiver stage of the CAN transceiver is built around a **two-stage hysteresis comparator**, which is essential for handling the noisy environment of a CAN bus. Since CAN lines often experience common-mode noise and electromagnetic interference, a simple comparator would generate unstable or jittery outputs, but a hysteresis comparator ensures stable switching.

The differential input signals from the bus, **VCANH and VCANL**, are fed into the first differential pair (transistors M42 and M43). The voltage difference between these signals represents the bus state (dominant or recessive). This differential input design enhances noise rejection because only the difference between the lines—not the absolute voltage—is evaluated.

Transistors M51 and M52 introduce **positive feedback** into the comparator, which creates a hysteresis window. This means that the input voltage must cross a higher threshold to switch from recessive to dominant (positive trip point) and a lower threshold to switch from dominant back to recessive (negative trip point). This design prevents rapid, unintended switching caused by small noise spikes.

The hysteresis window is about **0.4 V**, with a **positive trip point (V+TRP) of 0.9 V** and a **negative trip point (V-TRP) of 0.5 V**. These values ensure that common-mode noise and differential disturbances do not cause the receiver output to flicker or shift unpredictably. In noisy environments, such as automotive wiring looms, this significantly enhances communication reliability."

After the comparator, the output passes to a **buffer inverter** formed by **transistors M53 and M54**. The purpose of this buffer is twofold: it strengthens the output drive for the RXD pin and ensures that the voltage levels fully meet **TTL/CMOS logic standards**, providing a clean digital output for the CAN controller.

The receiver is also designed with excellent temperature stability. Simulations show that from $-40\text{ }^{\circ}\text{C}$ to $125\text{ }^{\circ}\text{C}$, the delay in the receiver output changes by only 18 ns, which is negligible at CAN speeds of 1 Mbit/s. This demonstrates that the comparator's threshold levels and switching behavior remain consistent across wide automotive temperature ranges.

Overall, the receiver stage is optimized to convert weak and noisy differential bus signals into a clean digital logic level while maximizing noise immunity, temperature stability, and compatibility with standard digital logic.

Symbol of all Stages:

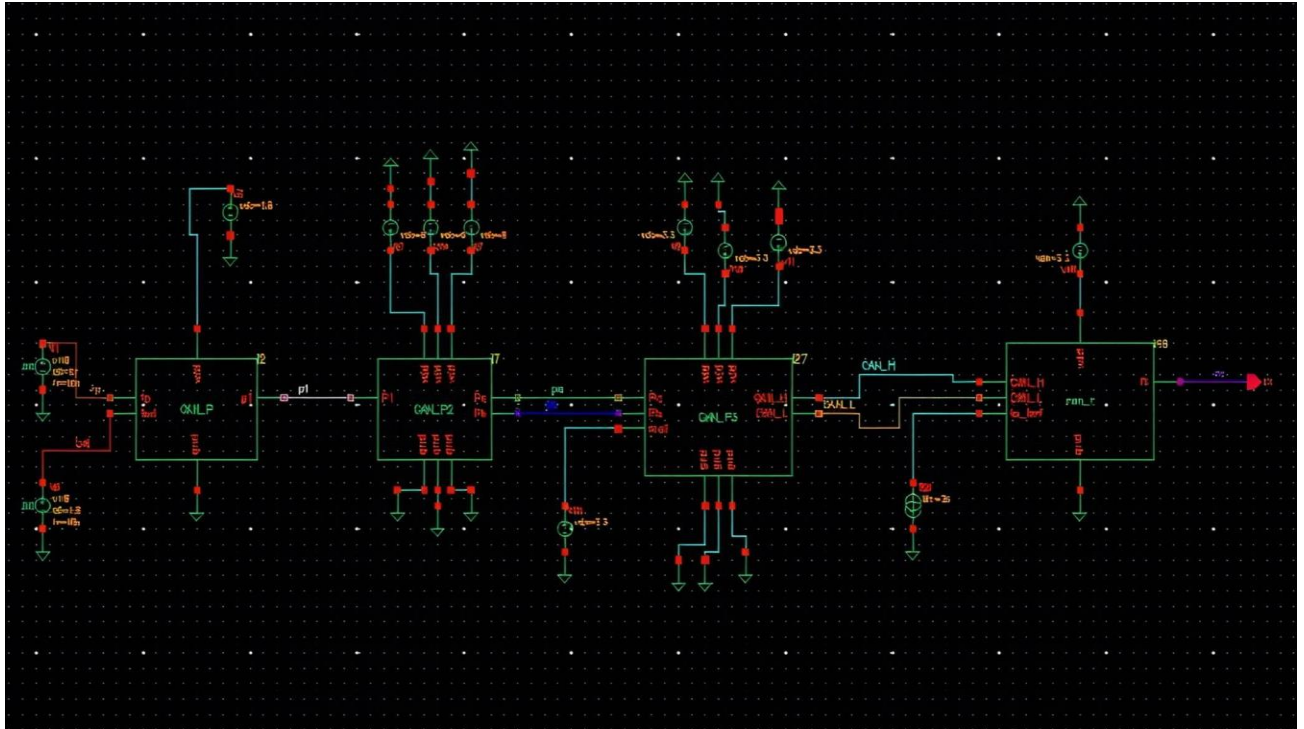


Figure 8.4.3: Symbol of all Stages

This schematic represents the simulation of a Controller Area Network (CAN) Transceiver System in Cadence Virtuoso, implemented using four main functional blocks labeled CAN_P, CAN_P2, CAN_P3, and CAN_R. Each block denotes a different stage of the CAN communication interface, and the surrounding voltage sources, grounds, and interconnections ensure proper biasing and operation of the system.

The rectangular symbols (CAN_P, CAN_P2, CAN_P3, CAN_R) represent hierarchical schematic blocks (subcircuits or modules):

- CAN_P – This is the transmitter block. It processes the digital input signal txd and converts it into internal control signals for further stages of the transceiver.
- CAN_P2 – This block works as the driver or buffer stage, providing signal strengthening and ensuring clean transitions between the transmitter and the output sections.
- CAN_P3 – This block forms part of the output driver stage. It helps generate the appropriate differential levels on CAN_H and CAN_L, contributing to setting the dominant (logic 0) or recessive (logic 1) states on the CAN bus.
- CAN_R – This is the receiver stage. It senses the differential bus voltages CAN_H and CAN_L, interprets the bus state, and converts it back into a digital output signal rx.
 - $CAN_H > CAN_L \rightarrow$ Dominant state (logic 0)
 - $CAN_H \approx CAN_L \rightarrow$ Recessive state (logic 1)

The small circular symbols labeled vdc indicate DC voltage sources (such as 1.8 V, 3.3 V, or 5 V) that supply the necessary power levels for the analog and digital components of each block.

Symbols marked gnd represent ground connections that provide a stable reference potential for the entire system.

Node labels such as P0, P1, P2, Pa, Pb, and vref interconnect the various stages, ensuring proper signal flow and simulation continuity across the transceiver.

Lastly, the CAN_H and CAN_L terminals act as the differential output lines of the CAN transceiver. These lines produce opposite-polarity signals required for reliable and noise-immune communication in the CAN protocol, enabling robust data transmission even in electrically harsh environments

CHAPTER 8

RESULTS AND DISCUSSION

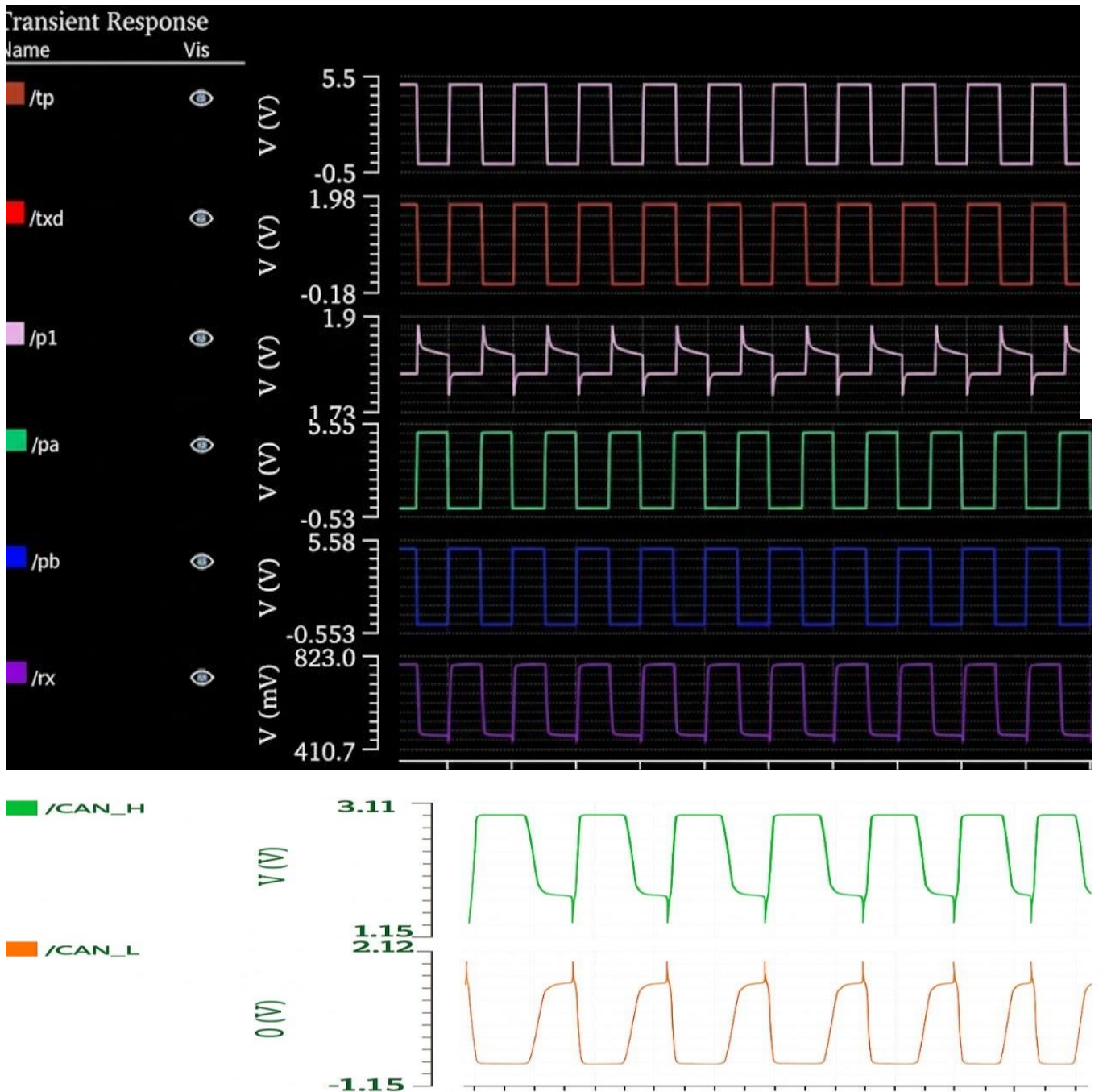


Figure 9.1: Waveform

1. /tp – Thermal Protection Signal

The /tp waveform appears as a square wave, indicating that thermal shutdown is being intentionally toggled during the simulation to verify the transmitter's protection response.

2. /txd – Transmit Data Input

This signal represents the digital input supplied by the CAN controller. It is a clean CMOS-level square wave that alternates between logic high and logic low. This waveform serves as the reference for evaluating the performance of the transmitter and verifying correct signal reconstruction at the receiver output.

3. /pa – Internal Driver Signal (Non-Inverting Path)

This internal node reflects the output of the transmitter's inverter chain. The signal closely follows the input TXD but incorporates the delay and shaping imposed by the cascaded inverters. It feeds one of the large output transistors in the driver stage and demonstrates proper amplification and timing.

4. /pb – Internal Driver Signal (Inverting Path)

The /pb waveform operates as the complement of /pa, being 180° out of phase. It drives the opposite branch of the differential output stage. The correct anti-phase relationship between /pa and /pb confirms that the push-pull output driver receives the appropriate control signals for differential CAN transmission

4.CAN_H – Differential Bus Output (High Line)

The CAN_H waveform represents the high-side output of the differential CAN driver. Together with CAN_L, it forms the differential signaling pair required by the ISO-11898 CAN physical layer. During a dominant bit, the driver actively pulls CAN_H toward a higher voltage while CAN_L is pulled lower, creating a strong differential voltage on the bus. During a recessive bit, both CAN_H and CAN_L return to their common-mode idle voltage, resulting in a small differential value.

5. /CAN_L – Differential Bus Output (CAN Low)

The CAN_L waveform demonstrates the proper generation of the differential bus signal. During dominant bits, CAN_L is driven toward a lower voltage, whereas during recessive bits it returns to a higher idle value. The transitions are shaped by the slew-rate control, giving the waveform gradual slopes that meet the CAN physical-layer requirements for EMI reduction. The signal maintains stability.

The phase difference between CAN_H and CAN_L signals also validates the correct operation of the differential output stage, ensuring that the two signals are perfectly 180° out of phase. This differential signaling enhances noise immunity because any external noise induced equally on both lines will be canceled out at the receiver end.

When the transceiver sends a dominant bit (logic 0), the voltage on CAN_H rises to approximately 3.1 V, while CAN_L drops to around 1.1 V, creating a differential voltage ($V_{diff} \approx 2 \text{ V}$) between the two lines. This differential voltage represents the active communication state on the CAN bus. Conversely, when the transmitter sends a recessive bit (logic 1), both lines settle to nearly the same potential (about 2.5 V), resulting in a zero differential voltage, indicating the idle state of the bus.

7. /rx – Receiver Output Signal

The receiver output /rx successfully reconstructs the digital input information from the differential bus signals. Despite the shaped (slew-limited) waveforms at the bus, /rx is clean, fast, and closely matches the original /txd input. This behavior results from the hysteresis comparator in the receiver, which suppresses noise and ensures stable switching. The close agreement between /txd and /rx verifies the correct end-to-end operation of the transceiver.

APPLICATION

1. Used in automotive electronics because it meets ISO-11898 standards and supports reliable in-vehicle communication. Its strong noise immunity makes it suitable for ECUs and safety-critical systems.
2. Suitable for industrial automation where harsh electrical environments exist. Its differential signaling and protection circuits ensure stable data transfer between controllers and sensors.
3. Applied in robotics systems that require dependable high-speed communication. The hysteresis comparator and strong EMI resistance help maintain signal integrity.
4. Useful in medical devices where compact, low-power communication interfaces are needed. The CMOS process reduces cost and power consumption, making it ideal for embedded medical systems.
5. Supports long-distance CAN networks as the adjustable slew rate helps reduce EMI, enabling stable signaling over extended cable lengths.

ADVANTAGES

1. Includes short-circuit, over-voltage, under-voltage, and thermal shutdown protection, improving reliability under fault conditions.
2. Exhibits high EMI immunity due to waveform symmetry and 0.4 V hysteresis, effectively filtering differential noise and reducing interference.
3. Uses low-cost CMOS technology, resulting in lower power consumption and making it economically suitable for large-scale applications.
4. Maintains stable performance over a wide -40°C to 125°C temperature range, ensuring dependable operation in harsh environments.

LIMITATIONS

1. Built using an older 180 nm CMOS process, which limits speed, integration, and power efficiency compared to modern technologies.
2. Limited to 1 Mbit/s CAN speed, making it unsuitable for newer CAN-FD applications that require significantly higher data rates.
3. Lacks modern low-power or standby modes, which increases energy consumption in battery-powered or idle systems.
4. Does not include advanced diagnostic features such as bus-fault detection or wake-up functions commonly found in newer transceivers.
5. Thermal shutdown at 150 °C may interrupt operation in extremely hot automotive situations, affecting availability under high load.

CONCLUSION

The design and simulation of the CMOS-based CAN transceiver demonstrate successful implementation of all essential functional blocks required for reliable CAN communication. The input stage correctly processes the TXD signal while incorporating thermal protection to ensure safe device operation under fault conditions. The middle-stage driver and output circuitry accurately generate the differential CANH and CANL signals, maintaining proper voltage levels and timing characteristics. The receiver stage, implemented with a hysteresis comparator, effectively reconstructs a clean digital RXD signal from the differential bus, providing strong noise immunity and stable operation even under disturbed conditions.

The simulation waveforms confirm that the transceiver performs end-to-end communication seamlessly, with accurate signal propagation from TXD to CANH/CANL and back to RXD. The proper operation of internal nodes, controlled edge shaping, and robust differential outputs ensure compliance with CAN physical-layer requirements. Overall, the results validate that the designed transceiver is reliable, thermally protected, noise-tolerant, and suitable for automotive and industrial CAN applications.

FUTURE SCOPE

1. Can be improved by migrating to smaller CMOS nodes like 65 nm or 45 nm for faster speed, lower power, and higher integration.
2. Future versions could support CAN-FD with higher data rates (5–8 Mbit/s), expanding compatibility with modern automotive networks.
3. Additional diagnostic and fault-tolerant features could be integrated to improve safety, including error detection and wake-up functionality.
4. Adding low-power standby or sleep modes would significantly reduce energy usage in systems with long idle periods.

References

1. H. Zhan, S. Zong, X. Han, and C. Li, "Design of the Circuits for a CMOS CAN Transceiver Chip with Slew Rate Control," *Advanced Materials Research*, vols. 433–440, pp. 1895–1902, Jan. 2012, doi: [10.4028/www.scientific.net/AMR.433-440.1895](https://doi.org/10.4028/www.scientific.net/AMR.433-440.1895).
2. A. S. Shinde, A. S. Deshpande, P. N. Kalamkar, and A. R. Nichal, "Implementation of CAN Bus Protocol," *International Journal on Recent and Innovation Trends in Computing and Communication (IJRITCC)*, vol. 2, no. 6, pp. 1568–1573, Jun. 2014.
3. N. Kim, D. Kim, K. Cho, J. Kim, and W. Cho, "Design and verification of a CAN controller for custom ASIC," in *Proceedings of the 11th International CAN Conference (iCC)*, CiA (CAN in Automation), 2005, pp. 09-13.
4. D. Batas and K. Schumacher, "An integrated analog-CMOS control system for low-cost applications," in *2002 9th International Conference on Electronics, Circuits and Systems (ICECS)*, Dubrovnik, Croatia, 2002, pp. 1239–1242, doi: 10.1109/ICECS.2002.1046254.
5. L. Ran, W. Junfeng, and W. Haiying, "Design Method of CAN BUS Network Communication Structure for Electric Vehicle," in *IFOST 2010 Proceedings*, Harbin, China, 2010, pp. 1–4, IEEE.
6. P. Kaur and C. K. Toor, "An FPGA Implementation of CAN Protocol in Automobiles," *Australian Journal of Basic and Applied Sciences*, vol. 11, no. 8 (Special), pp. 107–113, May 2017.

COURSE OUTCOMES

On completion of this course, students are able to:

1. Get exposed to literature available for research challenges in interdisciplinary fields.
2. Get an insight of real-world problems, challenges and related engineering issues to broader societal context.
3. Implement their idea using modern tools & new technologies.
4. Demonstrate professionalism with ethics, individual & team management and effective communication skills to comprehend & to write reports.

APPENDIX

1.CRC 15.v

```
// crc15.v - placeholder CRC15 block (for educational completeness)
// For this simplified example we omit a full CRC implementation.
// In production use, implement CRC-15 for CAN (polynomial 0x4599) or use a tested library.
module crc15(
    input wire clk,
    input wire rst_n,
    input wire en,
    input wire data_in,
    output reg crc_out
);
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) crc_out <= 0;
    else if (en) crc_out <= data_in; // placeholder behavior
end
endmodule
```

2.can bittiming.v

```
module can_bittiming #(
    parameter CLK_FREQ_HZ = 50000000
)(
    input wire    clk,
    input wire    rst_n,
    input wire [15:0] brp,    // prescaler (>=1)
    input wire [11:0] tseg_reg, // SJW[11:8], TSEG1[7:4], TSEG2[3:0]
    input wire    enable,
    output reg    bit_clk,    // pulses once per CAN nominal bit
    output reg    sample_point // asserted at sample point cycle
);
// Simple implementation: Use BRP to divide reference clock to TQ, and count TQs
reg [31:0] tq_cnt;
reg [31:0] tq_limit;
reg [7:0] tq_per_bit;
reg [7:0] tseg1, tseg2, sjw;
reg [7:0] tq_pos; // current tq position
reg [31:0] brp_cnt;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        brp_cnt <= 32'd0;
        tq_cnt <= 32'd0;
    end
end
```

```

    tq_per_bit <= 8'd16; // default
    tseg1 <= 8'd13;
    tseg2 <= 8'd2;
    sjw <= 8'd1;
    tq_pos <= 8'd0;
    bit_clk <= 1'b0;
    sample_point <= 1'b0;
end else begin
    // decode regs
    sjw <= tseg_reg[11:8];
    tseg1 <= tseg_reg[7:4];
    tseg2 <= tseg_reg[3:0];
    // simple safety defaults
    if (tseg1 == 8'd0) tseg1 <= 8'd13;
    if (tseg2 == 8'd0) tseg2 <= 8'd2;
    if (brp == 16'd0) tq_limit <= 32'd1; else tq_limit <= brp;
    if (!enable) begin
        brp_cnt <= 32'd0;
        tq_pos <= 8'd0;
        bit_clk <= 1'b0;
        sample_point <= 1'b0;
    end else begin
        // BRP divides reference clock into Time Quanta (TQ)
        if (brp_cnt >= (tq_limit-1)) begin
            brp_cnt <= 32'd0;
            if (tq_pos >= (tseg1 + tseg2 + 1)) begin
                tq_pos <= 8'd0;
                bit_clk <= 1'b1; // pulse
            end else begin
                tq_pos <= tq_pos + 8'd1;
                bit_clk <= 1'b0;
            end
            // sample point when tq_pos == tseg1 (after sync + prop)
            sample_point <= (tq_pos == tseg1);
        end else begin
            brp_cnt <= brp_cnt + 32'd1;
            bit_clk <= 1'b0;
            sample_point <= 1'b0;
        end
    end
end
end
endmodule

```

3.can tx.v

```
// can_tx.v - simplified transmitter (sends 8-bit data as a toy 'frame')
module can_tx(
input wire clk,
input wire rst_n,
input wire bit_clk,
input wire sample_point,
input wire [7:0] tx_data,
input wire tx_start,
output reg tx_busy,
output reg can_tx,
output reg tx_done
);
reg [3:0] state;
localparam IDLE=0, LOAD=1, SEND=2, EOF=3;
reg [3:0] bit_cnt;
reg [7:0] shift;
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
state <= IDLE; tx_busy <= 0; can_tx <= 1; tx_done <= 0;
bit_cnt <= 0; shift <= 0;
end else begin
tx_done <= 0;
if (tx_start && state==IDLE) begin
state <= LOAD; tx_busy <= 1;
end
if (sample_point) begin
case(state)
LOAD: begin
shift <= tx_data;
bit_cnt <= 0;
state <= SEND;
can_tx <= 1; // recessive before sending
end
SEND: begin
// send LSB first (toy format), dominant = 0, recessive = 1
can_tx <= shift[0];
shift <= {1'b1, shift[7:1]};
bit_cnt <= bit_cnt + 1;
if (bit_cnt==7) state <= EOF;
end
EOF: begin
```

```
can_tx <= 1; // release bus
tx_done <= 1;
tx_busy <= 0;
state <= IDLE;
end
default: state <= IDLE;
endcase
end
end
end
endmodule
```

4.can rx.v

// can_rx.v - simplified receiver to match the toy transmitter above

```
module can_rx(
input wire clk,
input wire rst_n,
input wire bit_clk,
input wire sample_point,
input wire can_rx,
output reg rx_valid,
output reg [7:0] rx_data,
output reg rx_done
);
reg [3:0] state;
localparam R_IDLE=0, R_RECV=1, R_DONE=2;
reg [3:0] bit_cnt;
reg [7:0] shift;
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
state <= R_IDLE;
bit_cnt <= 0;
shift <= 0;
rx_data <= 0;
rx_valid <= 0;
rx_done <= 0;
end else begin
rx_done <= 0;
rx_valid <= 0;
if (sample_point) begin
case(state)
R_IDLE: begin
```

```
// detect start by sampling: if bus != 1 (recessive) assume start
if (can_rx==1'b0) begin
state <= R_RECV;
bit_cnt <= 0;
shift <= 8'hFF;
end
end
R_RECV: begin
// sample at sample_point -- in this simplified model we sample on bit_clk
shift <= {can_rx, shift[7:1]};
bit_cnt <= bit_cnt + 1;
if (bit_cnt==7) state <= R_DONE;
end
R_DONE: begin
rx_data <= shift;
rx_valid <= 1;
rx_done <= 1;
state <= R_IDLE;
end
default: state <= R_IDLE;
endcase
end
end
end endmodule
```

5.can_tbu.v

```
// can_tbu.v - simplified bit timing uni
module can_tbu #(
parameter SYSCLK_FREQ = 100_000_000,
parameter BITRATE = 125_000
)(
input wire clk,
input wire rst_n,
output reg bit_clk,// pulses once per CAN bit-time
output reg sample_point
);
// asserted at sample point inside bit-time);
localparam integer CYCLES_PER_BIT = SYSCLK_FREQ / BITRATE;
integer counter;
integer sample_count;
```

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        counter <= 0;
        bit_clk <= 0;
        sample_point <= 0;
        sample_count <= 0;
    end else begin
        counter <= counter + 1;
        if (counter >= CYCLES_PER_BIT-1) begin
            counter <= 0;
            bit_clk <= 1;
            sample_count <= 0;
            sample_point <= 0;
        end
        else begin
            bit_clk <= 0;
            // sample at 75% of bit-time (simple)
            if (counter == (CYCLES_PER_BIT*3)/4) sample_point <= 1;
            else sample_point <= 0;
        end
    end
end
endmodule
```

6.can top.v

```
// can_top.v - Simplified CAN controller top
module can_top(
    input wire clk,// system clock
    input wire rst_n,
    // APB-lite/simple reg interface signals (simplified)
    input wire [7:0] tx_data,
    input wire tx_start,
    output wire tx_busy,
    output wire rx_valid,
    output wire [7:0] rx_data,
    // physical single-wire representation for simulation only
    output wire can_tx,
    // drives bus (0 dominant, 1 recessive)
    input wire can_rx
    // sampled bus
);
wire bit_clk;
```

```
wire sample_point;
wire tx_shift_en;
wire rx_shift_en;
wire [10:0] tx_frame_bits;
wire tx_done;
wire rx_done;
can_tbu #( .SYSCLK_FREQ(100_000_000), .BITRATE(125_000)) tbu (
    .clk(clk), .rst_n(rst_n),
    .bit_clk(bit_clk),
    .sample_point(sample_point)
);
can_tx txu (
    .clk(clk), .rst_n(rst_n),
    .bit_clk(bit_clk),
    .sample_point(sample_point),
    .tx_data(tx_data),
    .tx_start(tx_start),
    .tx_busy(tx_busy),
    .can_tx(can_tx),
    .tx_done(tx_done)
);
can_rx rxu (
    .clk(clk), .rst_n(rst_n),
    .bit_clk(bit_clk),
    .sample_point(sample_point),
    .can_rx(can_rx),
    .rx_valid(rx_valid),
    .rx_data(rx_data),
    .rx_done(rx_done)
);
Endmodule
```


7.tb_can.v(TestBench)

```
module tb_can;
    reg      clk;
    reg      rst_n;
    reg [7:0] tx_data;
    reg      tx_start;
    wire      tx_busy;

    wire      can_tx;
    wire      can_rx;

    wire      rx_valid;
    wire [7:0] rx_data;

    // LOOPBACK CONNECTION (can_rx <— can_tx)
    assign can_rx = can_tx;
    real can_h_v, can_l_v;
    always @(*) begin
        if (can_tx == 1'b0) begin
            can_h_v = 3.5;
            can_l_v = 1.5;
        end else begin
            can_h_v = 2.5;
            can_l_v = 2.5;
        end
    end
end

// DUT INSTANCE
can_top dut (
    .clk      (clk),
    .rst_n    (rst_n),
    .tx_data  (tx_data),
    .tx_start (tx_start),
    .tx_busy  (tx_busy),
    .rx_valid (rx_valid),
    .rx_data  (rx_data),
    .can_tx   (can_tx),
    .can_rx   (can_rx)
);

// CLOCK: 100 MHz → 10 ns period
always #5 clk = ~clk;
```

```
initial begin
    // DEFAULTS
    clk    = 0;
    rst_n  = 0;
    tx_start = 0;
    tx_data = 8'hA5;

    // RESET
    #100;
    rst_n = 1;

    // WAIT A LITTLE
    #200;

    // START TRANSMISSION (PULSE MUST BE WIDE ENOUGH)
    tx_start = 1;
    #50;      // MINIMUM 5 clock cycles for safety
    tx_start = 0;

    // WAIT FOR RECEIVER
    wait (rx_valid == 1);
    #20;

    $display("CAN RX DATA = %h at time %0t", rx_data, $time);

    #200;
    $finish;
end

endmodule
```

8.gedit

```
# =====  
# SDC for can_top (CAN TX/RX toy design)  
# =====  
  
# Units (optional, many tools set this by default)  
set_units -time ns -capacitance pF -voltage V -current mA  
  
# 1. Create main clock: 100 MHz on port clk (10 ns period)  
create_clock -name clk -period 10.000 -waveform {0 5} [get_ports clk]  
  
# 2. Optional: clock uncertainty (jitter + skew margin)  
set_clock_uncertainty 0.10 [get_clocks clk]  
  
# 3. Optional: input delays (external source drives inputs)  
#   Assume external device launches data on same clk  
#   and board delay  $\approx 1$  ns.  
set_input_delay -clock clk -max 1.0 [get_ports {rst_n tx_start tx_data[*]}]  
set_input_delay -clock clk -min 0.0 [get_ports {rst_n tx_start tx_data[*]}]  
  
# 4. Optional: output delays (towards external sink)  
#   Assume external device requires data 1 ns before its clock edge.  
set_output_delay -clock clk -max 1.0 [get_ports {tx_busy rx_valid rx_data[*] can_tx can_rx}]  
set_output_delay -clock clk -min 0.0 [get_ports {tx_busy rx_valid rx_data[*] can_tx can_rx}]  
  
# 5. Optional: simple design-rule constraints  
# (use small values suitable for 100 MHz)  
set_max_transition 0.5 [current_design]  
set_max_fanout 16 [current_design]
```

Contact Details

Project Associates	
Name: Amith A V USN: 4BD22EC007 Address: #3197/166, Mahalakshmi Layout, Near Kundwada Lake, Davangere -577006 E-mail: amithvenkatesh13@gmail.com Mobile Number: 7483994240	
Name: Chirag S USN: 4BD22E025 Address: #4122, 1st main, 2nd Cross, Bangla Badavane, Harihara E-mail: chiragchiru223@gmail.com Mobile Number: 6360214617	
Name: Mayuri S M USN: 4BD22EC057 Address: #141, Anjaneya Temple Road, Davanagere E-mail: mayurism16dvg@gmail.com Mobile Number: 7406382998	
Name: Rahul Pattar USN: 4BD22EC069 Address: Jaya Nivas, Near H V College, Harugeri, Raibag(Tq), Belagavi (Dist) E-mail: rahulpattar04@gmail.com Mobile Number: 8867597162	